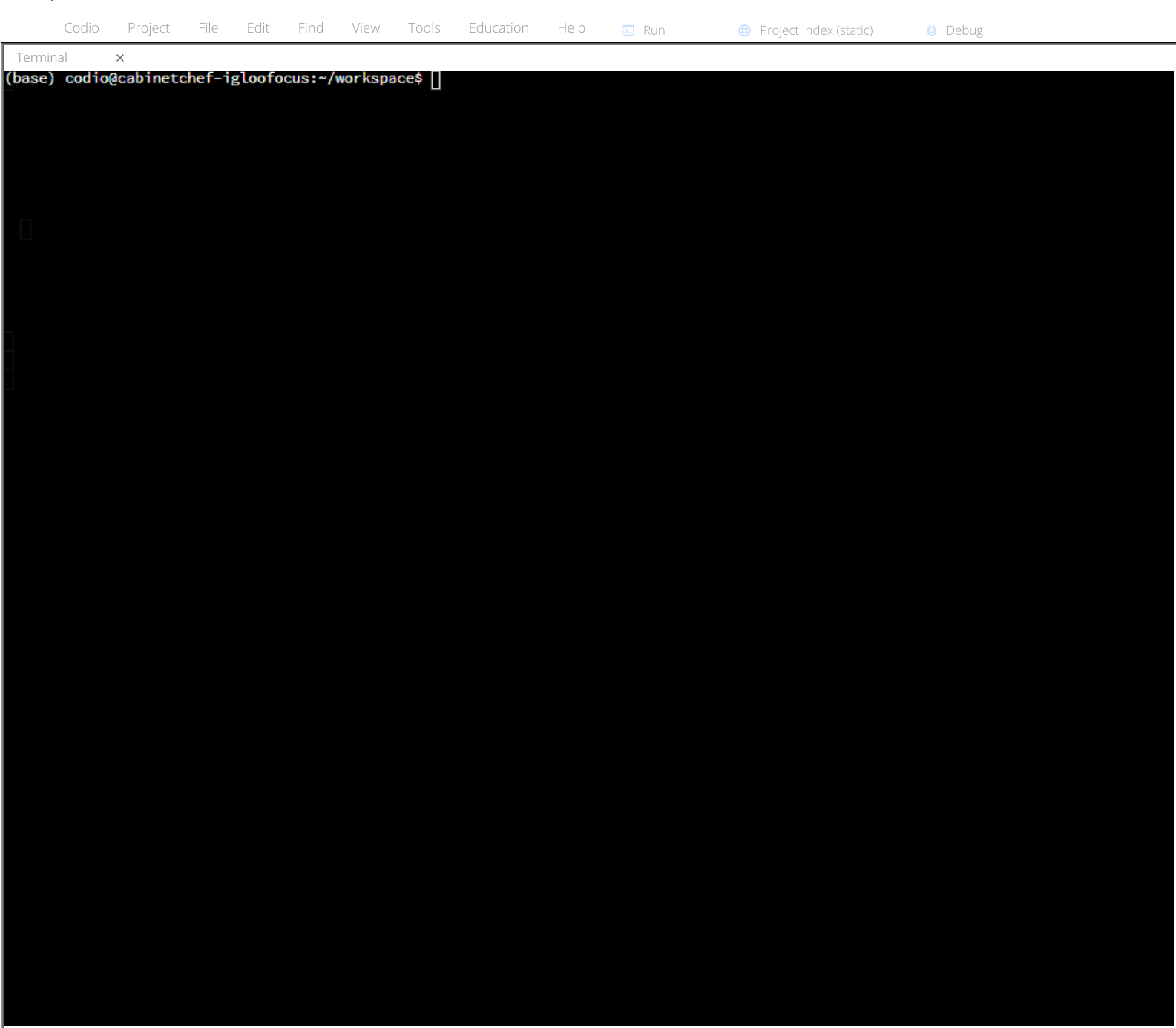




Code

Collapse

Testing Performance

Increased Performance (but not at first)

This time we are going to change into the `sw_characters/` directory and then activate the virtual environment. This means that when Python creates our `db.json` file, it will in the same directory as the modules.

```
cd sw_characters/ && conda activate sw
```

Use the string `"b"` as the search term. **Remember**, the database is going to be empty at first. So Python will have to do all of the API calls, which takes a long time.

```
python interface.py --name="b"
```

Now run the same exact search again:

```
python interface.py --name="b"
```

This time, it should be much faster. The results are not instantaneous, as the script still has to make the initial API call that generates the list of characters. After that, all of the information is pulled from the database. Every search for a new character search will be "slow" and saved to the database. Searching for these characters again will be much faster. Because the results are written to a file, they are persistent.

State of the Database

Let's take a look at how Python stores information in the database. There are two different views of this data. The first is the actual contents of the JSON file. Click the link below to open the database.

Notice how each record has a document ID. For example, Beru Whitesun has the ID `1`, while Biggs Darklighter has the ID `2`. **Important**, be careful about editing this file as it affects the database and the output of our script.

Open `db.json`

However, if you access the database through Python itself, you are presented with the same information in a slightly different format. Enter the command below to start the interactive Python shell. The prompt transforms to `>>>` when you are in the Python shell. We can enter Python code one line at a time and it will be executed when we press `enter`. Click on the tab with the terminal and start the Python shell.

```
python
```

First, import the `interface` module. Give it the alias `i` to reduce the amount of typing we need to do.

```
import interface as i
```

Now, take a look of the contents of the database object. Use the `i.all()` method to return the entire database. Remember, the database is located in the `interface` module which we refer to as `i`. You do not need to use a print statement as Python will send the return value to the interpreter automatically.

```
i.db.all()
```

Instead of seeing a large dictionary, Python returns a list of dictionaries. You are not presented with the document ID for each record. You can access and manipulate data like any other list in Python.

```
characters = i.db.all()
boba = characters[0]
boba['name'].upper()
```

Exit the Python shell with the following command:

```
exit()
```

Wrapping Up

To close out this project, we need to clean up our code a bit and prepare it for others to use.

- We no longer need the `parse_char_list` function. It is not hurting anything since we no longer call it, but removing it makes our code a bit easier to read and understand. This one is really up to you. Click the link below to open the `interface.py` file so you can remove the `parse_char_list` function (if you want).

Open `interface.py`

- As we prepare the project for use by others, we should remove the `db.json` file. Python will create this file if it is not present, so this won't break our script. In addition, this will reduce the size of our package when others download it. The user can populate the database with their own searches. Run the following command in the terminal:

```
rm db.json
```

- Finally we need to use Conda to create a requirements file in the `.yaml` format so that it includes all packages installed for this project. Should you share this project with others, they can use the `.yaml` file to build a virtual environment with all of the required packages with a single command.

```
conda env export > requirements.yaml
```

Verify that Conda created the `.yaml` file by using the `cat` command. This sends the contents of `requirements.yaml` to the terminal.

```
cat requirements.yaml
```

Verify that you see both the default and `conda-forge` channels listed in the document:

```
channels:
- conda-forge
- defaults
```

Also look for the packages installed with `pip`. That would be `fire` and any of its dependencies:

```
- pip:
- fire==4.8.0
- tqdm==4.49
- tornado==4.5.0
```

▼ Code

Your code should look like this:

- `search_api.py`

```
import requests

def search(search_term='luke'):
    base_url = 'https://swapi.dev/api/people/?search='
    search_url = f'{base_url}{search_term}'
    resp = requests.get(search_url)
    resp_json = resp.json()
    if resp_json.get('results'):
        return resp_json['results']
    else:
        return None

def parse_name(person):
    name = person.get('name')
    return name

def parse_planet(person):
    planet_url = person.get('homeworld')
    resp = requests.get(planet_url)
    planet = resp.json().get('name')
    return planet

def parse_files(person):
    file_urls = person.get('films')
    files = [fetch_title(file_url) for file_url in file_urls]
    return files

def fetch_title(url):
    file_json = requests.get(url).json()
    file_title = file_json.get('title')
    return file_title

def format_titles(titles):
    new_lines = [title + '\n' for title in titles]
    formatted_titles = ' '.join(new_lines)
    return formatted_titles

def person_description(name, planet, titles):
    description = f'{name} is from the planet {planet}. They appear in the following\n'
    return description

if __name__ == '__main__':
    import pprint

    person = search()
    name = parse_name(person)
    planet = parse_planet(person)
    file_list = parse_files(person)
    titles = format_titles(file_list)

    pprint(titles)


```

- `interface.py`

```
import fire
from typing import Typed, Query
import search_api

db = Typed[(db.json)]
User = Query()

def search(name='luke'):
    characters = search_api.search(name)
    if characters is not None:
        check_db(characters)
    else:
        print(f'Cannot find the character "{name}"')

def parse_char(char):
    char_name = search_api.parse_name(char)
    planet = search_api.parse_planet(char)
    file_list = search_api.parse_files(char)
    titles = search_api.format_titles(file_list)
    description = search_api.person_description(char_name, planet, titles)
    db.insert((name=char_name, 'planet'=planet, 'titles'=titles))
    return description

def check_db(char):
    for char in chars:
        char_name = search_api.parse_name(char)
        results = db.search(user_name=char_name)
        if not results:
            description = parse_char(char)
        else:
            name = results[0]['name']
            planet = results[0]['planet']
            titles = results[0]['titles']
            description = search_api.person_description(name, planet, titles)
        pprint(description)

if __name__ == '__main__':
    fire.Fire(search)


```

Deactivate the virtual environment.

```
conda deactivate
```

Lab Question

Assume you want to start a virtual environment called `project1` with Poetry. Fill in the blank below with the command that would accomplish this.

```
conda create --name project1
```

The correct answer is:

```
conda create --name project1
```

Use the `create` command followed by the `--name` flag. Then type the